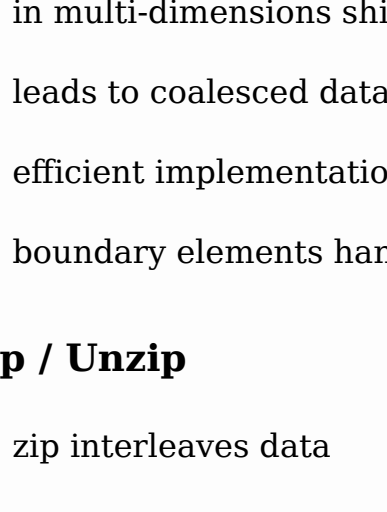


Data Reorganization Patterns

- data transfer is many times a bottleneck
- for data-intensive applications primary design focus should be on data movement, adding computation later
- shared-memory systems add additional cost
 - for efficient vectorization it is important to properly declare structures
 - effect of cache size on scalability (avoid false sharing)

Gather

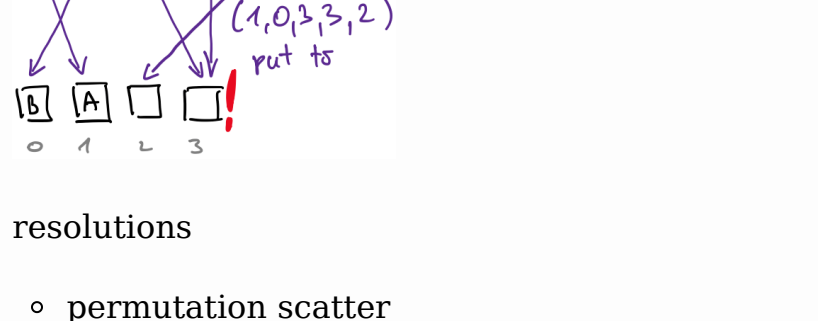
- gather collects all data from a collection of location indices and source arrays and places them into an output collection
- combination of a random read and map
- output data has
 - the same number of elements as the number of indices in input collections
 - the same dimensionality as location index collection



- MPI_Gather
 - less general
 - a lot can be gained with derived MPI data types

Shift / Rotate

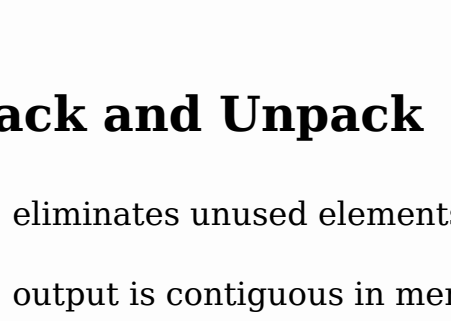
- special gathers
- have regular data access patterns



- can be efficiently implemented using vector instructions
- in multi-dimensions shift/rotate offsets may differ
- leads to coalesced data access
- efficient implementations using vector operations
- boundary elements handling

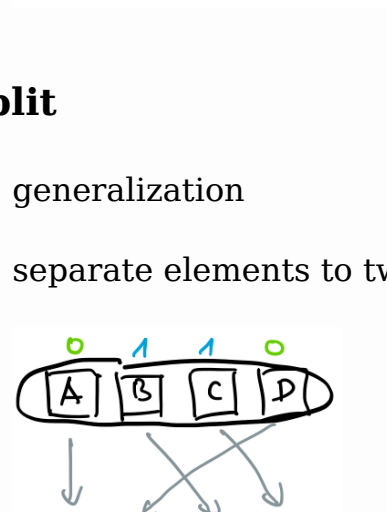
Zip / Unzip

- zip interleaves data
- example: assemble complex data by interleaving real and imaginary parts
- convert from structure of arrays to array of structures
- unzip reverses zip operation



Scatter

- a collection of input data is written to specified write locations
- multiple writes to the same location are possible



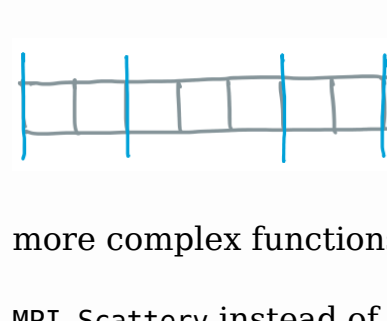
- resolutions
 - permutation scatter
 - collisions are illegal, array of indices should not have duplicates
 - can be always turned into gather when addresses are known in advance
 - example: matrix transpose
 - MPI_Scatter
 - merge scatter
 - combines values
 - only works with associative and commutative operators
 - example: histogram computation
 - atomic scatter
 - atomic writes, non-deterministic
 - can be deterministic when written input elements gave the same value
 - example: parallel disjunction (output array is initially cleared, writing true is actually OR operation)
 - priority scatter (deterministic using priorities)
 - priority based on a position of an element in input array
 - higher priority for elements at the end of the array is consistent with serial code

Gather vs Scatter

- scatter is more expensive
- gather reads versus scatter reads & writes (whole cache line); scatter on shared memory systems requires cores synchronization to keep cache coherent, false sharing may occur
- if addresses are known in advance, scatter can be converted to gather
- one option is also to scatter the addresses first and later gather data
- conversion takes resources
 - makes sense when it is used repeatedly
- suitable for shared-memory systems
- MPI_Gather and MPI_Scatter
 - optimized, not so general
 - no need for conversion

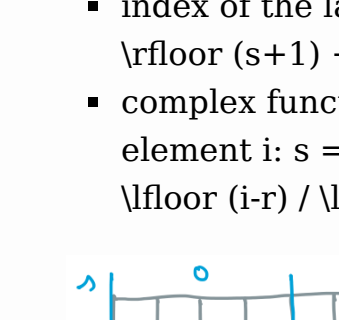
Pack and Unpack

- eliminates unused elements from collection
- output is contiguous in memory, which leads to better memory access and vectorization
- pack is combination of scan with conditional scatter
- pack can be fused with map
- useful when small number of elements is discarded



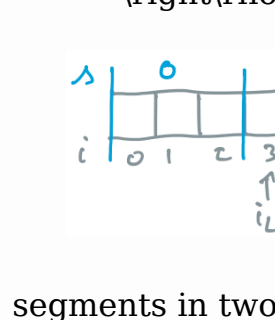
Split

- generalization
- separate elements to two or more sets



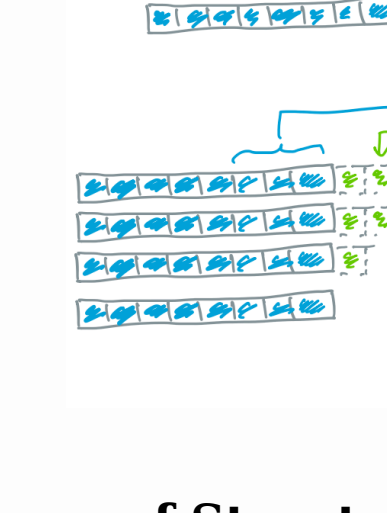
Expand

- in combination with map
- when map can produce arbitrary number of elements



MPI_Pack and MPI_Unpack

- packs a datatype into a contiguous memory
- useful for combining data of different data types to reduce number of sends
- MPI_Pack_size gives size of data in bytes; used to dynamically allocate size of pack structure
- copies data to new location (better to use data types)

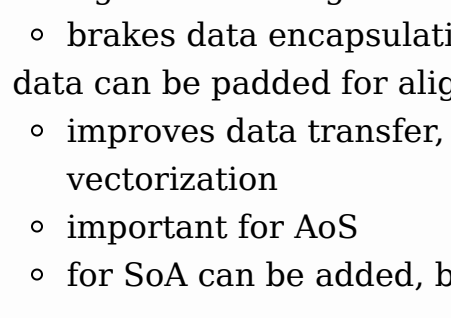


Geometric Decomposition

- common parallelization strategy
 - divide computational domain to sections
 - work on sections individually
 - combine the results
- geometric decomposition
 - spatially regular structure
 - image, grid, also sorting and graphs

Partition

- non-overlapping sections to avoid write conflicts and race conditions
- partitions are of equal size



- 1D or multi dimensions
- combined with map – no problems as it has exclusive access to partition
- can be further split to allow for nested (hierarchical) parallelism
- boundary elements require special treatment
 - partial sections along the edges, special code, but can be commonly parallelized/vectorized
- cache line size, vector-unit size
 - related to stencil strip-based operations

Segment

- like partition, but sections vary in size



- more complex functions for data manipulation must be used
- MPI_Scatterv instead of MPI_Scatter, ...
- segmentation along each dimension is possible (kD-tree)
- distributing N elements to S segments
 - larger-first approach
 - first $r = (n \bmod S)$ segments have one element more, $\lceil n/S \rceil$
 - other segments are of size $\lfloor n/S \rfloor$
 - index of the first element in segment s: $i_L = \lfloor n/S \rfloor s$
 - index of the last element in segment s: $i_H = \lfloor n/S \rfloor (s+1)$
 - complex function to determine to which segment belongs element i: $s = \min(\lfloor n/S \rfloor i / \lfloor n/S \rfloor + 1, \lfloor n/S \rfloor (i-r) / \lfloor n/S \rfloor + 1)$
 - mixed approach
 - larger and smaller segments are mixed
 - index of first element in segment s: $i_L = \lfloor n/S \rfloor s$
 - index of last element in segment s: $i_H = \lfloor n/S \rfloor (s+1)$
 - element i belongs to segment $s = \lfloor (i+1) / \lfloor n/S \rfloor \rfloor$



- segments in two dimensions
 - row-wise stripped
 - column-wise stripped
 - checkerboard
 - example: halo exchange
 - square matrix of size $N \times N$
 - exchange of edge elements between neighbouring segments
 - row stripped and column stripped: $2 \times N$
 - checkerboard: $4 \times \lceil N/\sqrt{S} \rceil$
- exchange of bordering elements
 - new cell value depends on the values of its neighbours
 - exchanging one element needed for next step of communication
 - exchanging k elements
 - exchange is needed only on every k-th step
 - some additional computation
 - latency hiding
 - initialization of communication cost more than some additional data transfer and computation



Array of Structures (AoS) vs Structures of Arrays (SoA)

- common data representation approach (AoS)
 - object-oriented programming
 - declare structures representing some object
 - vehicle has mass, position, velocity, acceleration, ...
 - create collection of that structure
 - vehicles can be presented as an array of data type vehicle
 - data is not aligned well for transfer, vectorization
 - nice for writing code, also beneficial when data is randomly read
- alternative approach (SoA)
 - for data transfer and vectorization, data layout may have to be modified for better performance
 - declare structure of collections
 - collection of masses, positions, velocities, accelerations, ...
 - data is now contiguous, better aligned
 - better way of representing data when majority of data is used

- conversion between AoS and SoA is not an easy task
 - significant changes in data structures
 - brakes data encapsulation
- data can be padded for alignment
 - improves data transfer, can be adjusted to cache line, simplifies vectorization
 - important for AoS
 - for SoA can be added, but is usually not really needed

